



MARIO RODRIGUEZ FERNANDEZ

9 de octubre, 2025

**Deteccion de anomalias simuladas en
trafico aereo real ADS-B mediante
Elastic Stack (SIEM)**

github.com/mariorfdez97/ADS-B_SIEM

Entrega final

Administracion de Sistemas

UO296136

Índice

1. Introduccion y justificacion	2
1.1. Alcance y objetivo de la segunda entrega	3
1.2. Estructura del documento	4
2. Fundamentos del Elastic Stack, aplicacion y relacion con la asignatura	4
2.1. De la teoria a la practica	5
3. Descripcion de la arquitectura y modulos del sistema	6
3.1. Vision general de la arquitectura	6
3.2. Servicio de adquisicion ADS-B	7
3.3. Co-ATC (backend y frontend)	7
3.4. Pipeline de ingesta y procesamiento (Logstash)	8
3.5. Elasticsearch	9
3.6. Kibana	9
3.7. Integracion SIEM en la interfaz Co-ATC	9
4. Implementacion practica del SIEM	9
4.1. Despliegue de Elastic Stack	10
4.2. Ingesta y normalizacion de datos ADS-B	11
4.3. Deteccion de anomalias de seguridad	13
4.4. Dashboards y visualizacion	14
4.5. Requisitos tecnicos y de calidad	16
5. Gestion del trafico ADS-B y flujo de datos	17
5.1. Adaptacion de Co-ATC al contexto de LEBL	17
5.2. Orquestador de trafico ADS-B	18
5.3. Implementacion del feed ADS-B (servicio adsb-feed)	19
5.4. Escenario operativo y control del ritmo de ingesta	21
5.5. Flujo completo y relacion con Elastic Stack	22
6. Ejemplo practico de funcionamiento y operacion	22
6.1. Escenario de ejemplo	22
6.2. Deteccion en Kibana (Discover)	24
7. Relacion con la asignatura y resumen de lo realizado	26
8. Conclusiones y pasos siguientes	27
9. Referencias y bibliografia	28

1. Introduccion y justificacion

La gestion de eventos e informacion de seguridad (SIEM) basada en Elastic Stack se ha consolidado como una solucion flexible y potente para centralizar logs, correlacionar datos y visualizar alertas en tiempo real, un conjunto de herramientas potente en entornos criticos como la aviacion. En la industria aeronautica se generan volúmenes masivos de datos (planificacion de vuelos, telemetria, mantenimiento, etc.), cuyo analisis en tiempo real es clave para mejorar la eficiencia operativa y la seguridad. Elastic Stack - conformado por Elasticsearch, Logstash y Kibana - permite almacenar y consultar grandes conjuntos de datos de forma distribuida, facilitando la deteccion de patrones anómalos y amenazas mediante busquedas en tiempo real y dashboards dinamicos.

Estas características han llevado a su adopcion en aviacion no solo para analisis operativos, sino tambien para reforzar la ciberseguridad: por ejemplo, Flightwatching, una empresa aeroespacial francesa, monitorea con esta tecnologia datos telemetricos de mas de 600 aeronaves en tiempo real, utilizando Kibana para tableros y alertas, e incluso deteccion de anomalias automatizada sobre dichos datos. Este caso demuestra la capacidad de Elastic Stack para centralizar datos aeronauticos y descubrir comportamientos fuera de lo normal en tiempo real, mejorando la conciencia situacional y la toma de decisiones en entornos de aviacion.

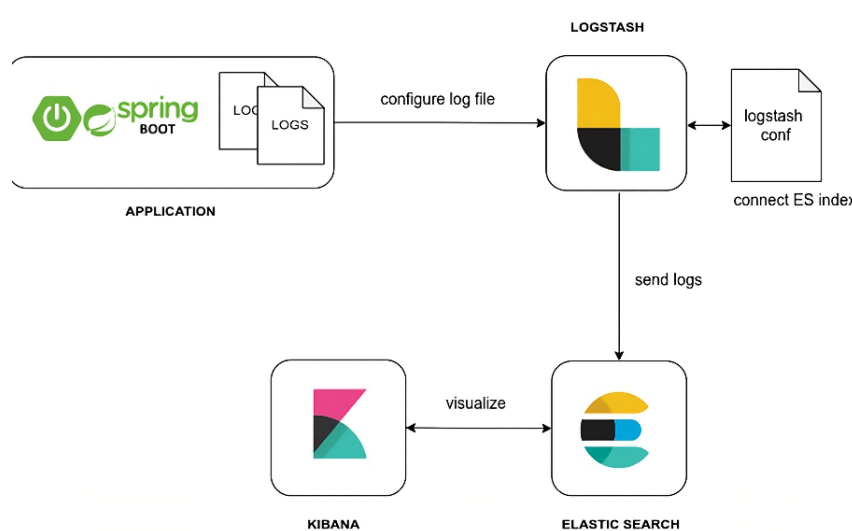


Figura 1. Ejemplo modular de arquitectura SIEM.

En este proyecto se aborda un caso concreto: la aplicacion de Elastic Stack como SIEM para detectar anomalias en datos de trafico aereo ADS-B simulado. ADS-B (*Automatic Dependent Surveillance-Broadcast*) es una tecnologia de vigilancia cooperativa en la cual las aeronaves emiten periodicamente por radio su posicion, velocidad, identificador, etc., proporcionando a los controladores informacion mas precisa y en tiempo real que los radares convencionales. Este sistema esta adoptado ampliamente a nivel internacional (en Espana la implantacion corre a cargo de ENAIRE).

No obstante, ADS-B carece de mecanismos de seguridad basicos: las transmisiones se emiten en abierto sin autentificacion ni cifrado, lo que las hace vulnerables a interferencias maliciosas. Un atacante con equipo radio y conocimiento del protocolo puede inyectar senales falsas en la

frecuencia (1090 MHz en aviacion comercial) simulando blancos inexistentes o distorsionando datos de aviones reales. Entre las amenazas documentadas se incluyen la suplantacion de aeronaves (*spoofing*) o la creacion de aviones fantasma (mensajes inventados que muestran trafico inexistente), aprovechando la ausencia de autentificacion.

Para reforzar la justificacion de este proyecto, resulta relevante senalar que en 2017 la FAA emitio alertas dirigidas al personal de mantenimiento, advirtiendole que las pruebas en tierra de sistemas ADS-B pueden generar «targets fantasma» en los radares de control de trafico aereo cuando se transmiten datos de posicion sin las precauciones adecuadas. Estos riesgos evidencian la necesidad de herramientas de monitorizacion que permitan distinguir actividad legitima de anomala en los datos ADS-B, alertando tempranamente sobre posibles incidentes de seguridad aerea.

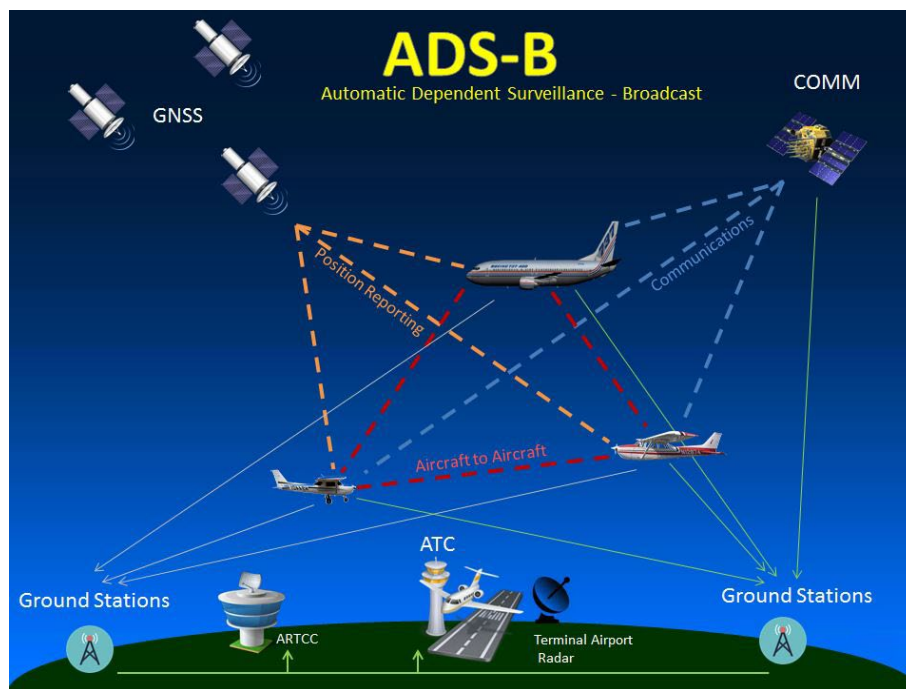


Figura 2. Funcionamiento ADS-B.

Un SIEM basado en Elastic Stack resulta idóneo para este cometido: puede correlacionar datos y aplicar reglas de detección para resaltar comportamientos atípicos, complementando la vigilancia tradicional de los controladores.

1.1. Alcance y objetivo de la segunda entrega

Esta segunda entrega traslada la propuesta teórica a una implementación real y reproducible. Se documenta la construcción del laboratorio con Docker, el flujo de datos ADS-B de extremo a extremo (feed → Co-ATC → Logstash → Elasticsearch → Kibana), las reglas SIEM aplicadas y la visualización operativa en dashboards. Además, se describe cómo se notifica al controlador en la interfaz de Co-ATC cuando se detectan anomalías.

El alcance es académico y controlado: se utiliza un laboratorio aislado y datos ADS-B públicos, sin cifrado ni autenticación, con anomalías simuladas para evaluar la detección. No se pretende

reemplazar sistemas ATC reales ni realizar certificaciones operativas; el foco esta en validar el modelo de ingesta, correlacion y visualizacion, y en mostrar como un SIEM puede aportar visibilidad y trazabilidad en un escenario critico.

1.2. Estructura del documento

El documento se organiza de forma incremental: primero se describe el despliegue y los componentes del stack, despues el proceso de ingesta y normalizacion, las reglas SIEM y los dashboards; finalmente se presenta un ejemplo practico de deteccion, la integracion de alertas en Co-ATC, las complicaciones encontradas y la relacion con el temario.

En resumen, el proyecto busca demostrar como la utilizacion de Elastic Stack como SIEM puede fortalecer la seguridad en entornos de aviacion civil, aportando visibilidad unificada sobre datos criticos y capacidad de respuesta ante anomalias, todo ello mediante una implementacion reproducible en un laboratorio academico.

2. Fundamentos del Elastic Stack, aplicacion y relacion con la asignatura

El Elastic Stack SIEM es un entorno avanzado de gestion de informacion y eventos de seguridad (SIEM) que integra diversos principios estudiados en la asignatura, combinando conocimientos de administracion de sistemas, monitorizacion de redes, gestion de logs, virtualizacion y seguridad de la informacion. Su implementacion practica en este proyecto no solo sirve como ejercicio tecnico, sino tambien como aplicacion directa de los fundamentos teoricos abordados en clase.

El Elastic Stack - compuesto principalmente por Elasticsearch, Logstash y Kibana - traduce estos conceptos teoricos en una arquitectura modular y escalable para el tratamiento de datos. A continuacion, se explican mas en detalle los modulos dichos anteriormente.

Elasticsearch

Elasticsearch funciona como el repositorio central donde se almacenan los mensajes ADS-B simulados. Su estructura de indices distribuidos permite buscar y analizar los datos en tiempo real, aplicando los principios de almacenamiento eficiente y gestion de registros de eventos estudiados en la asignatura.

Logstash

Se encarga del procesamiento intermedio y aplica conceptos de automatizacion de tareas y transformacion de flujos de datos, vistos en los temas de automatizacion de sistemas. Gracias a sus filtros y pipelines, muestra como crear un flujo controlado que convierte datos brutos en informacion estructurada y util.

Elastic Security (Kibana)

Integra los temas de visualizacion, analisis y gestion de incidentes, actuando como una interfaz que ayuda al administrador a interpretar datos y tomar decisiones con fundamento. Su modulo Elastic Security aplica directamente conceptos como deteccion de anomalias, correlacion de eventos y respuesta ante incidentes, tratados en la teoria sobre seguridad de la informacion y auditoria de sistemas.

Desde la vision del temario, el Elastic Stack SIEM tambien se apoya en los fundamentos de la virtualizacion y contenedorizacion de servicios. El proyecto utilizara Docker para desplegar de forma aislada cada componente del stack, lo cual refuerza el aprendizaje practico sobre gestion de entornos virtualizados, configuracion mediante contenedores y automatizacion de despliegues, permitiendo una administracion mas eficiente y reproducible del sistema.

En terminos de seguridad, la implementacion del SIEM ejemplifica los conceptos teoricos sobre deteccion de intrusiones y gestion de incidentes. Los mecanismos de correlacion de esta tecnologia permiten aplicar las tecnicas de deteccion basada en reglas y deteccion de anomalias estudiadas en el curso. En el contexto del proyecto, estas reglas se traducen en la identificacion de comportamientos anomalos en los datos ADS-B, como altitudes negativas o duplicaciones de identificadores ICAO.

Por otra parte, el uso de logs estructurados y analisis de datos en tiempo real conecta directamente con los temas del temario relacionados con administracion de registros del sistema (*syslog*, *journald*) y gestion de la informacion en entornos distribuidos. En el proyecto, la simulacion de trafico aereo genera flujos de datos continuos que son tratados como si provinieran de un entorno real de red o de infraestructura critica.

2.1. De la teoria a la practica

En la primera entrega se planteo Elastic Stack como una solucion SIEM aplicable a ADS-B. En esta implementacion, el enfoque se materializa en un laboratorio reproducible: los datos ADS-B se ingieren continuamente, se normalizan, se etiquetan con reglas de seguridad y se visualizan en Kibana. La correlacion no se limita a valores aislados (p. ej. velocidad alta), sino que incorpora consistencia fisica y contextual (RSSI vs distancia, altitud negativa, squawk de emergencia), lo que convierte el caso en un ejemplo realista de analisis de telemetria aeronautica.

El Elastic Stack SIEM actua como un puente entre la teoria y la practica de la asignatura. Integra en un entorno funcional todos los elementos que se abordan a lo largo del curso:

1. Principios de administracion de sistemas y servicios.
2. Virtualizacion y despliegue mediante contenedores.
3. Monitorizacion y analisis de logs.
4. Deteccion y gestion de incidentes de seguridad.
5. Automatizacion del control y respuesta ante eventos.

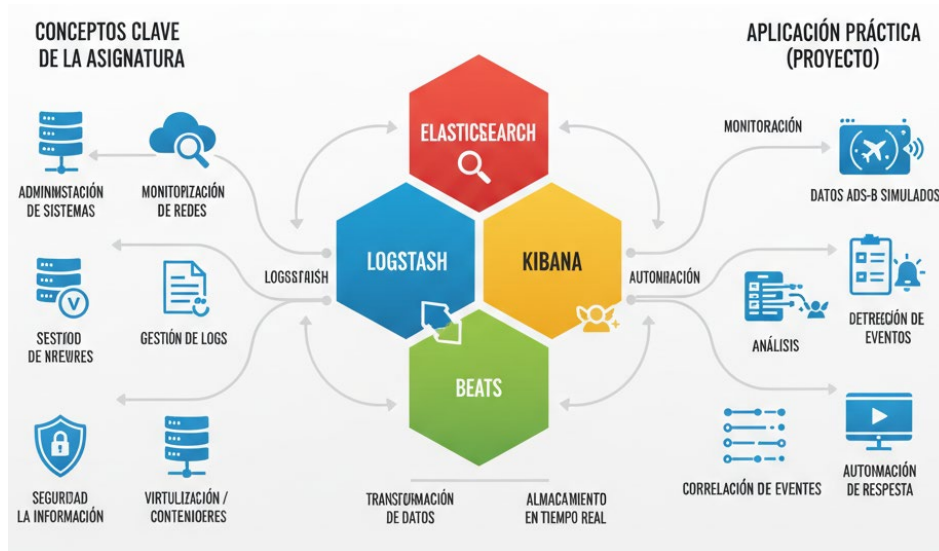


Figura 3. Relacion entre Elastic Stack y la asignatura.

3. Descripción de la arquitectura y modulos del sistema

El sistema desarrollado se basa en una arquitectura modular y distribuida, diseñada para integrar la adquisicion de datos ADS-B reales, su visualizacion operativa y su analisis mediante un pipeline SIEM basado en Elastic Stack. Cada modulo cumple una funcion bien definida dentro del flujo global, facilitando la mantenibilidad, la trazabilidad de los datos y la posibilidad de ampliacion futura.

3.1. Vision general de la arquitectura

La arquitectura del laboratorio se organiza en torno a cinco bloques principales:

- Servicio de adquisicion ADS-B.
- Backend y frontend Co-ATC.
- Pipeline de ingesta y procesamiento (Logstash).
- Almacenamiento y analisis (Elasticsearch).
- Visualizacion y explotacion (Kibana e interfaz de alertas en Co-ATC).

Todos los componentes se despliegan mediante contenedores Docker y se comunican a traves de una red interna, garantizando un entorno reproducible y aislado.

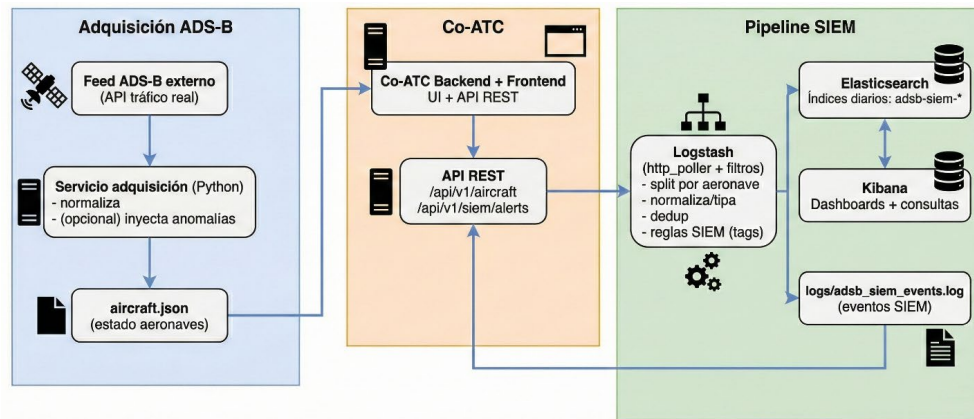


Figura 4. Descripción modular general SIEM.

3.2. Servicio de adquisición ADS-B

El servicio de adquisición ADS-B es el encargado de obtener datos de tráfico aéreo real desde una fuente externa y actuar como punto de entrada del sistema. Está implementado en Python y consulta periódicamente la API ADS-B externa con un intervalo configurable, lo que permite controlar el ritmo de ingesta y garantizar la estabilidad del laboratorio.

Los datos recibidos se filtran y normalizan, seleccionando únicamente los campos relevantes y adaptándolos al formato utilizado por Co-ATC. De forma opcional, el servicio permite la inyección controlada de anomalías, registrando dichas alteraciones para su posterior trazabilidad.

Finalmente, el estado consolidado de las aeronaves se almacena en un fichero `aircraft.json`, que es usado directamente por Co-ATC.

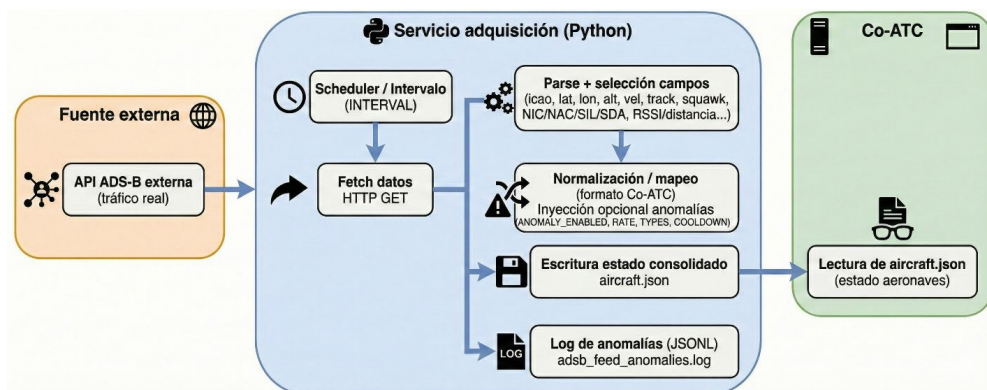


Figura 5. Arquitectura interna y flujo del servicio de adquisición ADS-B.

3.3. Co-ATC (backend y frontend)

Co-ATC actúa como núcleo: lee el fichero `aircraft.json`, mantiene el estado interno de las aeronaves y ofrece una visualización en tiempo casi real mediante una interfaz web. Además, expone una API REST que permite a otros módulos acceder al estado de los vuelos y a las alertas SIEM.

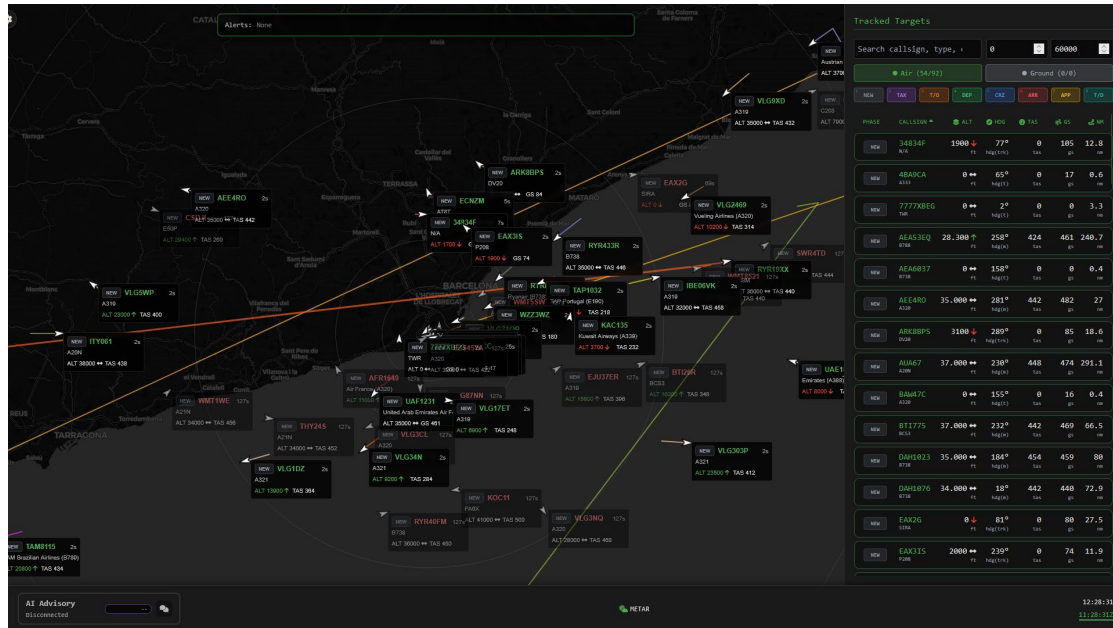


Figura 6. Panel de control de trafico aereo (Co-ATC) y monitorizacion de aeronaves.

3.4. Pipeline de ingesta y procesamiento (Logstash)

Logstash actua como el nucleo del sistema SIEM, siendo responsable de la ingesta, procesamiento y enriquecimiento de los datos ADS-B expuestos por Co-ATC. Este componente consulta periodicamente la API REST interna de Co-ATC y transforma la informacion recibida en eventos individuales por aeronave.

Durante el procesamiento, Logstash normaliza y tipa los campos ADS-B, calcula metricas derivadas y elimina estados redundantes mediante mecanismos de deduplicacion. Ademas, aplica reglas de deteccion de anomalias basadas en umbrales y relaciones fisicas entre parametros de vuelo, etiquetando los eventos anormales con marcas SIEM.

Los eventos resultantes se envian a Elasticsearch para su almacenamiento y analisis, y de forma complementaria se registran en logs estructurados locales, que son utilizados para la visualizacion de alertas en la interfaz de Co-ATC. Este enfoque permite una deteccion temprana de anomalias y una integracion directa con los sistemas de visualizacion.

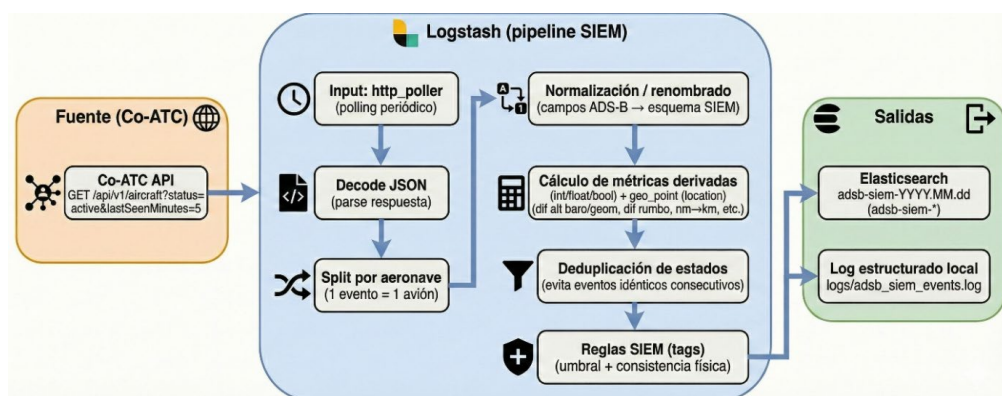


Figura 7. Pipeline de Logstash para el procesamiento y enriquecimiento de eventos.

3.5. Elasticsearch

Elasticsearch proporciona el almacenamiento persistente y el motor de búsqueda y agregación. Los eventos ADS-B se indexan en índices diarios, permitiendo consultas eficientes, análisis temporal y geoespacial, y la construcción de visualizaciones avanzadas.

3.6. Kibana

Kibana se utiliza como herramienta principal de visualización y análisis. A partir de los índices generados en Elasticsearch, se construyen dashboards, gráficos y consultas que permiten monitorizar el tráfico aéreo y detectar patrones anómalos de forma visual e intuitiva.

3.7. Integración SIEM en la interfaz Co-ATC

Como complemento a Kibana, el sistema incorpora un módulo de alertas integrado directamente en la interfaz de Co-ATC. Este módulo consulta eventos SIEM almacenados en logs estructurados y presenta alertas en tiempo casi real al usuario, cerrando el ciclo de detección y visualización operativa.

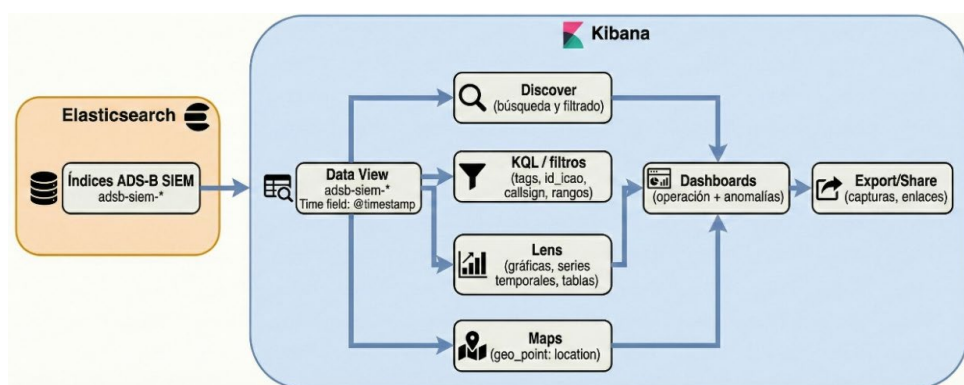


Figura 8. Flujo de visualización y explotación de datos en Kibana.

4. Implementación práctica del SIEM

En esta segunda entrega, el objetivo principal es trasladar la propuesta teórica a una implementación real del SIEM con Elastic Stack. Esto implica definir requisitos claros, demostrar cómo se cumplen en el laboratorio y dejar trazabilidad entre el diseño y la ejecución. El apartado se centra en lo estrictamente técnico: ingestión de datos ADS-B, transformación y correlación en Logstash, indexación en Elasticsearch, visualización en Kibana y notificación en la interfaz Co-ATC.

Para ordenar la implementación, se definen requisitos funcionales y no funcionales. Los funcionales garantizan que el sistema opere de extremo a extremo; los no funcionales aseguran estabilidad, rendimiento y reproducibilidad.

Cuadro 1. Requisitos funcionales del sistema.

Codigo	Descripcion
R1	Ingestion periodica de trafico ADS-B en formato estandarizado.
R2	Inyeccion controlada de anomalias para simular ataques o condiciones anomalas.
R3	Correlacion SIEM y etiquetado automatico de eventos.
R4	Visualizacion y analisis en Kibana mediante dashboards.
R5	Notificacion de alertas en la interfaz de Co-ATC.

Cuadro 2. Requisitos no funcionales del sistema.

Codigo	Descripcion
RN1	Reproducibilidad mediante contenedores Docker.
RN2	Control del ritmo de ingesta y limites de frecuencia.
RN3	Trazabilidad de eventos mediante logs estructurados.
RN4	Separacion modular de servicios para facilitar el mantenimiento.

4.1. Despliegue de Elastic Stack

El laboratorio se despliega mediante Docker Compose, lo que permite aislar cada componente y reproducir el entorno completo en cualquier maquina. Esta aproximacion refuerza la modularidad del sistema y facilita tanto el arranque como la depuracion. El archivo `docker-compose.yml` define los servicios principales del stack (Elasticsearch, Logstash y Kibana), junto con los servicios especificos del proyecto (adsb-feed y Co-ATC).

El despliegue se basa en una arquitectura por contenedores conectados en la misma red interna de Docker. Cada servicio expone puertos especificos al host para permitir el acceso local (Kibana en 5601, Elasticsearch en 9200, Co-ATC en 8000, etc.), y utiliza volúmenes para persistir datos o compartir ficheros entre servicios.

```

1 version: "3.8"
2
3 services:
4   adsb-feed:
5     build:
6       context: ./services/adsb-feed
7     restart: unless-stopped
8     environment:
9       - ANOMALY_ENABLED=true
10      - ANOMALY_RATE=0.01
11      - ANOMALY_MAX_PER_WRITE=1
12      - ANOMALY_COOLDOWN_CYCLES=30
13      - ANOMALY_RSSI_DISTANCE_RATE=0.01
14      - ANOMALY_RSSI_DISTANCE_MAX_PER_WRITE=1
15      - ANOMALY_TYPES=altitud_negativa,velocidad_excesiva,salto_posicion,squawk_emergencia,posible_spoofing
16      - ANOMALY_LOG=/logs/adsb-feed_anomalies.log
17      - INTERVAL=3
18     volumes:
19       - aircraft_data:/data
20       - ./logs:/logs
21     ports:
22       - "9000:9000"
23
24   co-atc:
25     build:
26       context: ./co-atc-main
27     restart: unless-stopped
28     depends_on:
29       - adsb-feed
30     environment:
31       - COATC_CONFIG=/app/configs/config.docker.toml
32     volumes:
33       - ./co-atc-main/configs/config.docker.toml:/app/configs/config.docker.toml:ro
34       - coatc_data:/app/data
35       - ./logs:/logs:ro
36     ports:
37       - "8000:8000"
38
39   elasticsearch:
40     image: docker.elastic.co/elasticsearch/elasticsearch:8.14.1
41     restart: unless-stopped
42     environment:
43       - discovery.type=single-node
44       - xpack.security.enabled=false
45       - ES_JAVA_OPTS=-Xms1g -Xmx1g
46     ulimits:
47       memlock:
48         soft: -1
49         hard: -1
50       nofile:
51         soft: 65536
52         hard: 65536
53     ports:
54       - "9200:9200"
55     volumes:
56       - esdata:/usr/share/elasticsearch/data
57     healthcheck:
58       test: ["CMD-SHELL", "curl -fs http://localhost:9200/_cluster/health || exit 1"]
59       interval: 10s
60       timeout: 5s
61       retries: 10
62
63   kibana:
64     image: docker.elastic.co/kibana/kibana:8.14.1
65     restart: unless-stopped
66     depends_on:
67       elasticsearch:
68         condition: service_healthy
69     environment:
70       - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
71       - XPACK_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY=0123456789abcdef0123456789abcdef
72       - XPACK_SECURITY_ENCRYPTIONKEY=0123456789abcdef0123456789abcdef
73       - XPACK_REPORTING_ENCRYPTIONKEY=0123456789abcdef0123456789abcdef
74     ports:
75       - "5601:5601"
76
77   logstash:
78     image: docker.elastic.co/logstash/logstash:8.14.1
79     restart: unless-stopped
80     depends_on:
81       elasticsearch:
82         condition: service_healthy
83       co-atc:
84         condition: service_started
85     user: root
86     command: ["-w", "-i"]
87     environment:
88       - LS_JAVA_OPTS=-Xms512m -Xmx512m
89     volumes:
90       - ./infra/logstash/pipeline:/usr/share/logstash/pipeline:ro
91       - ./logs:/logs
92     ports:
93       - "5044:5044"
94       - "9600:9600"
95     healthcheck:
96       test: ["CMD-SHELL", "curl -fs http://localhost:9600/_node/pipelines || exit 1"]
97       interval: 10s
98       timeout: 5s
99       retries: 10
100
101   volumes:
102     esdata:
103     coatc_data:
104     aircraft_data:
105

```

Figura 9. Extracto de la configuracion docker-compose.yml.

4.2. Ingesta y normalizacion de datos ADS-B

Una vez desplegada la infraestructura base, el objetivo se centra en la implementacion de una ingesta real, continua y normalizada de datos ADS-B, basada en trafico aereo real y no en eventos simulados mediante archivos de log estaticos, tal y como se planteo inicialmente en la primera entrega.

En la implementacion final, la fuente principal de datos es un feed ADS-B real, obtenido mediante un servicio auxiliar que consulta periodicamente una API externa de trafico aereo y genera un fichero `aircraft.json` con el estado actualizado de las aeronaves. Este fichero es consumido directamente por el backend Co-ATC, que actua como punto central de agregacion y exposicion de la informacion ADS-B a traves de una API HTTP REST.

De este modo, la ingesta SIEM se apoya en el estado operativo real mostrado al usuario en

Co-ATC, garantizando coherencia entre la visualizacion y los datos analizados.

La canalizacion de datos en la implementacion final se gestiona de la siguiente manera:

3. **Co-ATC como fuente de ingesta logica.** El backend de Co-ATC actua como agregador central del trafico ADS-B. Su endpoint `GET /api/v1/aircraft` devuelve periodicamente la lista de aeronaves activas con todos los campos necesarios para el analisis (posicion, altitud, velocidad, senal, integridad, etc.). De este modo, Co-ATC sustituye al simulador propuesto en la primera entrega y se convierte en la fuente unica de verdad del sistema.
4. **Logstash como motor de ingesta y procesamiento.** Logstash (`/infra/logstash.conf`) utiliza un input `http\_poller` para consultar periodicamente la API de Co-ATC. La respuesta JSON se divide en eventos individuales (uno por aeronave), se normaliza y se enriquece con campos derivados. Este enfoque elimina la necesidad de Filebeat (planteado inicialmente) y simplifica la arquitectura al trabajar con una fuente estructurada en tiempo real.



```
1 input {
2   http_poller {
3     urls => {
4       "co_atc_api" => {
5         method => "get"
6         url => "http://co-atc:8000/api/v1/aircraft?status=active&lastSeenMinutes=5"
7         headers => {
8           "Accept" => "application/json"
9         }
10      }
11    }
12    request_timeout => 10
13    schedule => { every => "5s" }
14    codec => "json"
15    ecs_compatibility => "v8"
16    tags => ["co-atc-api"]
17  }
18 }
```

Figura 10. Configuracion del `http\_poller` en Logstash.

5. **Elasticsearch como almacenamiento y motor analitico.** Los eventos procesados se indexan en Elasticsearch bajo indices diarios (`adsb-siem-YYYY.MM.dd`). Los campos se tipan correctamente (numericos, temporales y geoespaciales), permitiendo busquedas eficientes, agregaciones temporales y visualizacion geografica en Kibana.

Con este flujo, el sistema opera en tiempo casi real, con una latencia minima entre la actualizacion del estado en Co-ATC y su disponibilidad en Elasticsearch, dashboards de Kibana y alertas SIEM.

```
1 # 3. RENOMBRAMIENTO MASIVO (Nuevos campos añadidos)
2 mutate {
3   rename => {
4     "[aircraft][hex]" => "id_icao"
5     "[aircraft][flight]" => "callsign_vuelo"
6     "[aircraft][adsb][r]" => "matricula_aeronave"
7     "[aircraft][adsb][t]" => "modelo_aeronave"
8     "[aircraft][airline]" => "aerolinea"
9
10    "[aircraft][status]" => "estado"
11    "[aircraft][on_ground]" => "en_tierra"
12    "[aircraft][phase][current][0][phase]" => "fase_vuelo"
13    "[aircraft][date_tookoff]" => "hora_despegue"
14    "[aircraft][date_landed]" => "hora_aterrizaje"
15
16    "[aircraft][adsb][lat]" => "latitud"
17    "[aircraft][adsb][lon]" => "longitud"
18    "[aircraft][adsb][track]" => "rumbo_real"
19    "[aircraft][adsb][mag_heading]" => "rumbo_magnetico"
20    "[aircraft][distance]" => "distancia_receptor_nm"
21
22    "[aircraft][adsb][alt_baro]" => "altitud_pies"
23    "[aircraft][adsb][alt_geom]" => "altitud_geometrica_pies"
24    "[aircraft][adsb][baro_rate]" => "tasa_vertical_pies_min"
25    "[aircraft][adsb][nav_altitude_mcp]" => "altitud_objetivo_piloto"
26
27    "[aircraft][adsb][gs]" => "velocidad_tierra_nudos"
28    "[aircraft][adsb][ias]" => "velocidad_indicada"
29    "[aircraft][adsb][tas]" => "velocidad_verdadera"
30    "[aircraft][adsb][mach]" => "numero_mach"
31    "[aircraft][adsb][roll]" => "angulo_alabeo"
32
33    "[aircraft][adsb][rssi]" => "intensidad_senal_rssi"
34    "[aircraft][adsb][nic]" => "integridad_navegacion_nic"
35    "[aircraft][adsb][nac_p]" => "precision_posicion_nacp"
36    "[aircraft][adsb][squawk]" => "codigo_transpondedor"
37    "[aircraft][adsb][category]" => "categoria_estela"
38  }
39 }
```

Figura 11. Renombrado masivo y normalizacion de campos ADS-B.

4.3. Deteccion de anomalias de seguridad

Una vez los datos ADS-B son ingeridos, normalizados e indexados, la deteccion de anomalias de seguridad en la implementacion final se realiza durante la fase de procesamiento en Logstash, en lugar de apoyarse en el modulo Elastic Security de Kibana, tal y como se propuso inicialmente en la primera entrega.

Este cambio de enfoque responde a una decision de diseno orientada a simplificar el pipeline, mantener la deteccion estrechamente ligada a la ingesta de datos y facilitar la integracion directa de alertas en la interfaz operativa de Co-ATC. De este modo, la deteccion de anomalias se aproxima a un modelo de SIEM basado en reglas en tiempo de ingestion, ampliamente utilizado en entornos de monitorizacion y analisis de logs.

El objetivo de este subsistema es identificar, clasificar y etiquetar comportamientos anomalos o potencialmente maliciosos en las transmisiones ADS-B, que puedan indicar errores en los datos recibidos, inconsistencias fisicas o posibles intentos de spoofing.

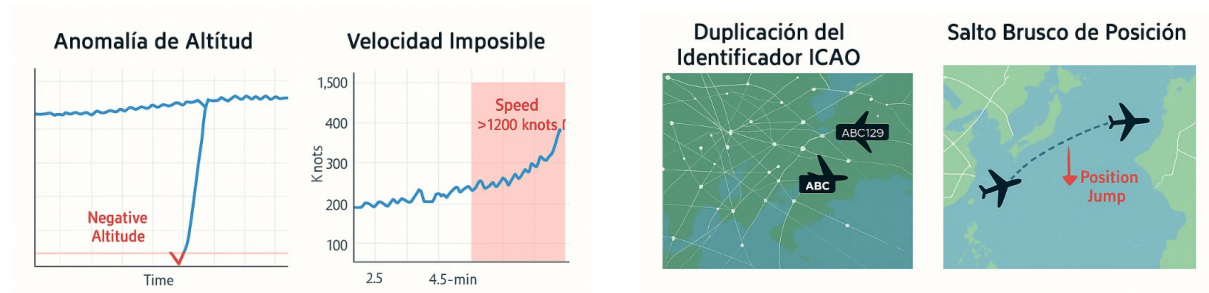


Figura 12. Ejemplos visuales de posibles anomalías.

- Altitudes negativas o fuera de rango operativo, que pueden indicar errores de sensor, datos corruptos o estados incoherentes.
- Velocidades imposibles, superiores a umbrales físicamente razonables para aeronaves convencionales.
- Incoherencias entre altitud barométrica y geométrica, que sugieren errores de referencia o degradación de la calidad de navegación.
- Tasas verticales extremas o inestables, incompatibles con maniobras normales de vuelo.
- Incoherencias entre intensidad de señal (RSSI) y distancia al receptor, patrón comúnmente asociado a datos falsificados o recepción anómala.
- Códigos squawk de emergencia (7500, 7600, 7700), tratados como alertas críticas de seguridad operacional.

Cada una de estas anomalías se implementa como una regla de detección independiente dentro de la pipeline de Logstash, mediante filtros condicionales que evalúan los campos del evento ADS-B una vez normalizados. Cuando se cumple una condición de detección, el evento es etiquetado con una o varias banderas SIEM (*tags*) que identifican el tipo de anomalía detectada.

4.4. Dashboards y visualización

Una parte esencial del proyecto es la creación de dashboards interactivos en Kibana, que sirvan como interfaz visual para la monitorización del tráfico aéreo y la gestión de alertas. Estos paneles permitirán al analista observar tanto la actividad operativa del espacio aéreo simulado como las incidencias de seguridad detectadas por el SIEM.

Los dashboards principales incluirán lo siguiente.

Creación de Data View y campo temporal

Antes de construir visualizaciones, es imprescindible crear un *data view* que apunte a los índices generados por el SIEM. En Kibana se realiza desde *Stack Management* → *Data Views* → *Create data view*. En este proyecto se utiliza el patrón `adsb-siem-*`, ya que Logstash indexa en Elasticsearch con formato diario (`adsb-siem-YYYY.MM.dd`).

El campo temporal recomendado es `@timestamp`, que Logstash genera a partir de

`aircraft.last\_seen`, lo que permite que las búsquedas y graficos se ordenen por el instante real de detección. Una vez creado el *data view*, es importante ejecutar *Refresh field list* para que Kibana detecte todos los campos y tipos.

Como verificación rápida, se recomienda comprobar que los campos críticos aparecen con el tipo adecuado:

- `altitud\_pies`, `velocidad\_tierra\_nudos`, `intensidad\_senal\_rssi` como numéricos.
- `latitud` y `longitud` como numéricos.
- `tags` como array de texto.

Además, es recomendable fijar un rango temporal inicial (por ejemplo, «Últimos 15 minutos») y guardar una búsqueda base para reutilizarla en varios paneles.



```
1      dist_nm = event.get('distancia_receptor_nm')
2      if dist_nm
3        event.set('distancia_receptor_km', dist_nm.to_f * 1.852)
4      end
5
6      v_tierra = event.get('velocidad_tierra_nudos')
7      if v_tierra && v_tierra.to_f > 1200
8        event.tag('alerta_velocidad_excesiva')
9      end
10
11     alt = event.get('altitud_pies')
12     ias = event.get('velocidad_indicada')
13     if alt && ias && alt.to_f < 10000 && ias.to_f > 260
14       event.tag('alerta_velocidad_baja_cota')
15     end
16
17     if alt && alt.to_f < -100
18       en_tierra = event.get('en_tierra')
19       if en_tierra == false || en_tierra.nil?
20         event.tag('alerta_altitud_negativa')
21       end
22     end
```

Figura 13. Configuración del *Data View* en Kibana para la indexación de eventos.

Consultas y filtros KQL recomendados

KQL permite filtrar eventos en *Discover* y en dashboards de forma rápida y legible. En este proyecto se utiliza para aislar tipos de anomalías, aeronaves concretas o rangos físicos (altitud, velocidad, RSSI). Es importante recordar que KQL distingue tipos numéricos, texto y arrays.

Ejemplos básicos

- Buscar todas las alertas: `tags: "alerta_*`
- Solo alertas de velocidad excesiva: `tags: "alerta\_velocidad\_excesiva"`
- Filtrar una aeronave concreta por ICAO: `id\_icao: "4BCE15"`

- Filtrar por *callsign* parcial: `callsign\_vuelo: "IBE*"`

Ejemplos numericos y combinados

- Altitud fisicamente imposible: `altitud\_pies < -100`
- Velocidad extrema: `velocidad\_tierra\_nudos > 1200`
- Spoofing por RSSI y distancia incoherente: `intensidad\_senal\_rssi >= -5 and distancia\_receptor\_nm >= 200`
- Baja integridad de navegacion: `integridad\_navegacion\_nic < 6`

Operadores utiles

- `and`, `or`, `not` para combinar condiciones.
- Parentesis para agrupar: `tags: ("alerta\_velocidad\_excesiva" or "alerta\_altitud\_negativa")`
- Existencia de campo: `rssi:*` o `tags:*`

En dashboards, estos filtros se pueden aplicar como *filter pills* para cada panel, o guardar una busqueda en *Discover* y reutilizarla como fuente de datos. Esto permite definir paneles especializados (p. ej., «solo emergencias» o «solo spoofing»), y mantener consistencia entre visualizaciones.

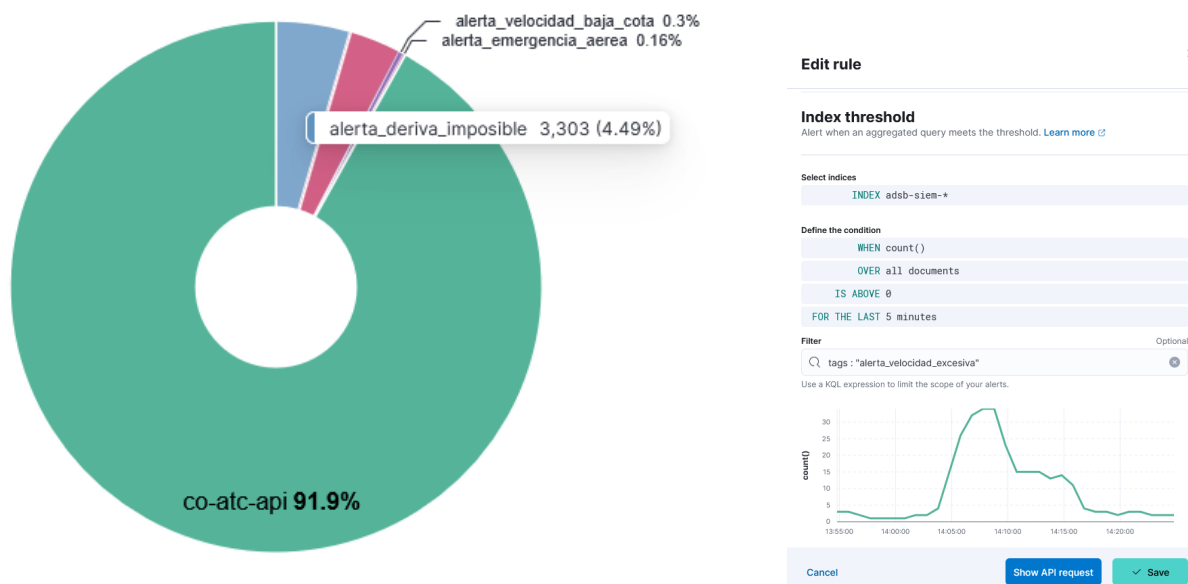


Figura 14. Distribucion porcentual de alertas SIEM y configuracion de una regla de deteccion basada en umbrales (*Index threshold*) en Kibana.

4.5. Requisitos tecnicos y de calidad

El sistema debera cumplir los siguientes requisitos globales para asegurar su correcto funcionamiento:

- **Integridad de datos:** la ingesta debe ser determinista y consistente. Logstash debe procesar cada evento una sola vez y conservar la coherencia entre el estado mostrado en Co-ATC y el estado indexado en Elasticsearch.
- **Fiabilidad:** los contenedores deben poder reiniciarse de forma independiente sin perder datos persistentes (volumenes), y Logstash debe reintentar envios cuando Elasticsearch no este disponible temporalmente.
- **Usabilidad:** Kibana debe proporcionar dashboards claros con mapas y series temporales, permitiendo filtrar por ICAO, rango temporal y tipo de alerta.
- **Portabilidad:** todo el entorno debe ser reproducible con un unico `docker-compose up`, garantizando consistencia entre desarrollo y laboratorio.

5. Gestion del trafico ADS-B y flujo de datos

Para construir un entorno de ingesta de trafico aereo que resultara didactico y tecnicamente manejable, se decidio partir del proyecto *open-source* Co-ATC, el cual ya funciona como backend de monitorizacion aeronautica. De este modo, se reduce el tiempo invertido en aspectos ajenos a los objetivos principales del proyecto y se focaliza el trabajo en la administracion y explotacion del SIEM.

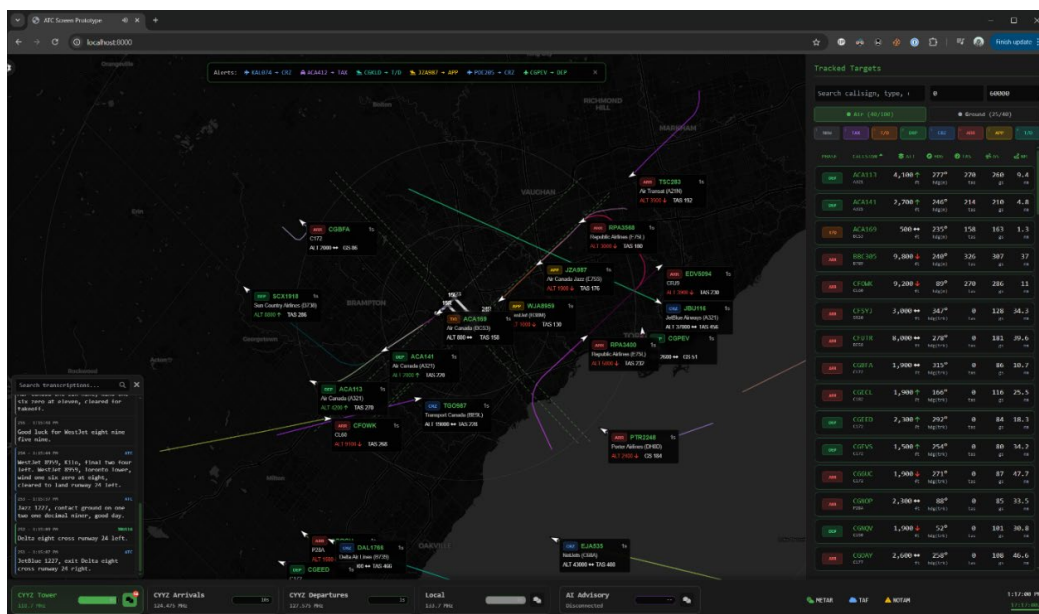


Figura 15. Menu principal de Co-ATC.

5.1. Adaptacion de Co-ATC al contexto de LEBL

Co-ATC es un proyecto *open-source* que permite visualizar, gestionar y simular aeronaves mediante una interfaz web y una API REST. Co-ATC puede funcionar tanto con receptores ADS-B reales como con fuentes locales de datos JSON, lo que lo convierte en una herramienta ideal para entornos educativos o de desarrollo sin necesidad de hardware aeronautico.

El programa se entrega con una configuracion orientada a Toronto (CYYZ). Para que los calculos de distancia, rumbo y filtros visuales sean coherentes con un escenario espanol, se modifiko `co-atc-main/configs/config.toml`. El fragmento siguiente resume el cambio mas relevante.



Figura 16. Ajustes de configuracion para Co-ATC y fichero minimo `aircraft.json`.

2. `local\_source\_url`: ahora apunta a `http://127.0.0.1:9000/aircraft.json`, un feed que servimos con `python3 -m http.server` para que Co-ATC tenga siempre un fichero disponible.
4. `[station]`: define la latitud, longitud, elevacion y el codigo ICAO de Barcelona. Con ello, las herramientas de Co-ATC calculan correctamente distancias a la estacion, fases de vuelo y filtros geograficos en la interfaz.

Ademas, se anadio un fichero minimo `co-atc-main/data/aircraft.json`. Este JSON vacio evita errores cuando el backend intenta leer un feed inexistente. Al servirlo mediante un HTTP local (ejemplo: `python3 -m http.server 9000` dentro de `co-atc-main/data/`), garantizamos que el bucle de ingesta se mantenga activo y que las aeronaves inyectadas se integren en la vista web y en los historicos de SQLite.

5.2. Orquestador de trafico ADS-B

En la implementacion final del proyecto, el concepto de orquestador de trafico simulado planteado en la primera entrega es sustituido por un orquestador de trafico ADS-B real, basado en la ingestion periodica de datos procedentes de una fuente externa de trafico aereo operativo.

En lugar de generar aeronaves de forma sintetica mediante un script de simulacion, el sistema emplea un servicio auxiliar de ingesta ADS-B, cuya funcion principal es consultar de forma periodica una API externa de trafico aereo y transformar la informacion recibida al formato esperado por el backend Co-ATC. Este servicio actua como intermediario entre la fuente ADS-B real y el sistema de visualizacion y analisis.

5.3. Implementacion del feed ADS-B (servicio adsb-feed)

El feed se implementa en `services/adsb-feed/feed.py`. Este servicio consulta de forma periodica la API publica <https://opendata.adsb.fi/api/v3> y transforma los resultados al formato `aircraft.json` esperado por la API Co-ATC. El proceso es ciclico: en cada ciclo descarga el estado actual de todas las aeronaves, normaliza los campos (altitud, velocidad, posicion, senal, etc.) y escribe el fichero actualizado en `/data/aircraft.json`.

La normalizacion se realiza en el propio feed para asegurar consistencia entre lo que consume Co-ATC y lo que finalmente indexa Logstash. Ademas, el feed puede inyectar anomalias controladas (si se activan) mediante variables de entorno (`ANOMALY\*`). Esto permite simular ataques de spoofing, altitudes imposibles o velocidades extremas sin alterar la logica del backend.



```
1 # Coordenadas y radio de consulta.
2 lat = float(os.environ.get("LAT", "41.2971"))
3 lon = float(os.environ.get("LON", "2.0785"))
4 dist = float(os.environ.get("DIST", "80"))
5
6 # Intervalo en segundos entre peticiones.
7 interval = float(os.environ.get("INTERVAL", "2"))
8
9 # Ruta donde se escribe aircraft.json.
10 out_path = Path(os.environ.get("OUT", "/data/aircraft.json"))
11 out_path.parent.mkdir(parents=True, exist_ok=True)
12
13 # Log donde guardamos anomalias (JSONL).
14 anomaly_log_path = Path(os.environ.get("ANOMALY_LOG", "/data/adsb_feed_anomalies.log"))
15 anomaly_log_path.parent.mkdir(parents=True, exist_ok=True)
16
17 # Anomalias activas o no.
18 anomaly_enabled = os.environ.get("ANOMALY_ENABLED", "false").strip().lower() in {
19     "1",
20     "true",
21     "yes",
22     "y",
23     "on",
24 }
```

Figura 17. Configuracion por entorno del servicio adsb-feed.

1. **Configuracion por entorno.** El feed se parametriza con variables de entorno para hacerlo flexible.
2. **Normalizacion de datos ADS-B.** El resultado es un *dict* limpio con claves estandar (`hex`, `flight`, `alt_baro`, `gs`, `rsi`, etc.).
3. **Selección de anomalías.** `pick_indices(count, max_count, rate, seed)` selecciona que aeronaves serán modificadas ese ciclo. Funciona así:
 - Evalúa cada índice con una pseudo-probabilidad.
 - Respeta `ANOMALY_RATE` y `ANOMALY_MAX_PER_WRITE`.
 - Cambia cada 5 segundos para evitar patrones repetitivos.

Esto permite control fino de la «densidad» de anomalías. `pick_type(types, seed, hex_code)` elige el tipo de anomalía a aplicar de forma estable. Si no se definen tipos, usa un valor por defecto.

```

1 def map_aircraft(ac: Dict[str, Any]) -> Dict[str, Any]:
2     """Normaliza un avion del feed externo al formato esperado por co-atc."""
3     alt_baro = safe_alt(ac.get("alt_baro"))
4     alt_geom = safe_alt(ac.get("alt_geom", ac.get("alt_baro")))
5     gs = ac.get("gs") or ac.get("tas") or ac.get("ias")
6     heading = ac.get("true_heading") or ac.get("track") or ac.get("mag_heading")
7     baro_rate = ac.get("baro_rate") or ac.get("geom_rate")
8     return {
9         "hex": (ac.get("hex") or "").upper(),
10        "type": ac.get("type"),
11        "flight": (ac.get("flight") or "").strip(),
12        "r": ac.get("r"),
13        "t": ac.get("t"),
14        "desc": ac.get("desc"),
15        "alt_baro": alt_baro,
16        "alt_geom": alt_geom,
17        "gs": gs,
18        "tas": ac.get("tas"),
19        "ias": ac.get("ias"),
20        "mach": ac.get("mach"),
21        "track": ac.get("track"),
22        "true_heading": heading,
23        "mag_heading": ac.get("mag_heading"),
24        "baro_rate": baro_rate,
25        "geom_rate": ac.get("geom_rate"),
26        "squawk": ac.get("squawk"),
27        "emergency": ac.get("emergency"),
28        "category": ac.get("category"),
29        "lat": ac.get("lat"),
30        "lon": ac.get("lon"),
31        "nic": ac.get("nic"),
32        "rc": ac.get("rc"),
33        "seen_pos": ac.get("seen_pos"),
34        "version": ac.get("version", 2),
35        "nic_baro": ac.get("nic_baro"),
36        "nac_p": ac.get("nac_p"),
37        "nac_v": ac.get("nac_v"),
38        "sil": ac.get("sil"),
39        "sil_type": ac.get("sil_type"),
40        "gva": ac.get("gva"),
41        "sda": ac.get("sda"),
42        "alert": ac.get("alert"),
43        "spi": ac.get("spi"),
44        "mlat": ac.get("mlat", []),
45        "tisb": ac.get("tisb", []),
46        "messages": ac.get("messages"),
47        "seen": ac.get("seen"),
48        "rssi": ac.get("rssi"),
49    }

```

Figura 18. Normalizacion y mapeo de aeronaves en map_aircraft.

4. **Inyeccion de anomalias.** inject_anomaly(ac, anomaly_type, seed) aplica modificaciones segun el tipo. Cada tipo esta ajustado para cruzar umbrales SIEM reales:

- altitud_negativa → alt_baro < -100
- velocidad_excesiva → gs > 1200
- posible_spoofing → rssi > -10 y nic < 6, ademas de nac_p, nac_v, sil y sda bajos.
- squawk_emergencia → 7500/7600/7700
- salto_posicion → lat/lon desplazados dentro de limites validos

Al final de cada anomalia, se fuerza tambien el patron RSSI + distancia incoherente:

- RSSI fuerte (-5 a -1).
- Se desplaza lat/lon para simular distancia > 200 NM.

Esto asegura que Logstash detecte alerta_rssi_distancia_incoherente.

5. **Logging de anomalias.** Cada anomalia se guarda en JSON Lines (uno por avion) en ANOMALY_LOG. Esto deja trazabilidad de que campo fue alterado y como.

A screenshot of a code editor window with a black border and three colored window control buttons (red, yellow, green) in the top-left corner. The editor contains Python code for logging anomalies. The code is as follows:

```
1 if hex_code:
2     last_anomaly_cycle[hex_code] = cycle_counter
3     modified_hexes.add(hex_code)
4     try:
5         with anomaly_log_path.open("a", encoding="utf-8") as fh:
6             detailed_changes = []
7             for field, values in changes.items():
8                 detailed_changes.append(
9                     {
10                        "field": field,
11                        "correct_value": values[0],
12                        "modified_value": values[1],
13                    })
14             record = {
15                 "ts": ts,
16                 "id_icao": item.get("hex"),
17                 "anomaly_type": anomaly_type,
18                 "changes": detailed_changes,
19             }
20             fh.write(json.dumps(record) + "\n")
21     except Exception:
22         # Si falla el log, seguimos igual.
23         pass
```

Figura 19. Registro estructurado de anomalías generadas en el feed.

6. **Bucle principal.** El ciclo principal hace:

- a) GET a <https://opendata.adsb.fi/api/v3/lat/<lat>/lon/<lon>/dist/<dist>>
- b) Mapeo de aeronaves con `map\_aircraft`
- c) Inyección de anomalías (si está activado)
- d) Escritura de `aircraft.json`
- e) *Sleep* por `INTERVAL`

El resultado es un `aircraft.json` vivo y actualizado que Co-ATC consume como fuente ADS-B local. En entorno Docker, la URL usada es `http://adsb-feed:9000/aircraft.json` (red interna de contenedores).

5.4. Escenario operativo y control del ritmo de ingesta

El laboratorio opera sobre un escenario basado en tráfico ADS-B real, eliminando la generación de vuelos simulados con rutas y horarios predefinidos. Co-ATC refleja en tiempo casi real el estado de las aeronaves visibles en la zona de interés, proporcionando un entorno más realista para el análisis SIEM.

Para garantizar la estabilidad del sistema, el servicio de adquisición ADS-B incorpora mecanismos de control del ritmo de ingesta, ajustando la frecuencia de consulta al feed externo y la actualización del estado de las aeronaves. Este control evita sobrecargas y mantiene una latencia reducida entre la recepción del dato y su disponibilidad en el pipeline Elastic.

Adicionalmente, el procesamiento en Logstash aplica deduplicación de estados consecutivos, reduciendo eventos redundantes sin perder información relevante. De forma opcional, el sistema permite la inyección controlada de anomalías para la validación de reglas SIEM, manteniendo siempre un comportamiento estable del escenario operativo.

5.5. Flujo completo y relacion con Elastic Stack

En funcionamiento normal, Co-ATC mantiene el estado actualizado de las aeronaves a partir del flujo ADS-B real y lo expone mediante el endpoint `/api/v1/aircraft`. Este endpoint constituye la fuente de datos para el pipeline SIEM del laboratorio.

Logstash consulta periodicamente dicha API y procesa la informacion recibida, separando los datos por aeronave y aplicando tareas de normalizacion, deduplicacion y deteccion de anomalias. Los eventos resultantes se indexan en Elasticsearch en indices diarios, permitiendo su almacenamiento y analisis historico.

Kibana utiliza estos indices para la creacion de dashboards, visualizaciones y consultas orientadas tanto a la monitorizacion del trafico aereo como a la identificacion de comportamientos anormales. De forma complementaria, los eventos etiquetados como anomalias se registran en logs estructurados, que son consumidos directamente por la interfaz de Co-ATC para mostrar alertas operativas en tiempo casi real.

Este flujo integra de forma coherente Co-ATC y Elastic Stack, proporcionando un entorno SIEM funcional y alineado con los objetivos del laboratorio.

6. Ejemplo practico de funcionamiento y operacion

Este apartado describe un caso de uso real dentro del laboratorio para mostrar como se detecta una anomalia, como se visualiza en Kibana y como se notifica al controlador en Co-ATC. El objetivo es conectar la teoria con la operacion diaria del sistema.

6.1. Escenario de ejemplo

Para mostrar el funcionamiento real del laboratorio, se describe un escenario operativo completo en el que una anomalia es inyectada, detectada y visualizada por el sistema SIEM. El objetivo de este ejemplo es demostrar la coherencia del flujo entre el feed ADS-B, el backend de Co-ATC, Logstash, Elasticsearch y Kibana, y como esa deteccion llega finalmente al controlador aereo (ATC).

Para iniciar el laboratorio desde la raiz del repositorio, lo primero es levantar todos los servicios con Docker Compose. El comando recomendado es `docker-compose up -d -build`.

```

mario on 12:48:54 ~master
mario ~$ sudo docker-compose up -d
[sudo] password for mario:
Creating network "ads-b-siem_default" with the default driver
Creating ads-b-siem_elasticsearch_1 ... done
Creating ads-b-siem_adsb-feed_1 ... done
Creating ads-b-siem_co-atc_1 ... done
Creating ads-b-siem_logstash_1 ... done
Creating ads-b-siem_kibana_1 ... done

mario on 12:49:24 ~master
mario ~$ sudo docker-compose ps

```

Name	Command	State	Ports
ads-b-siem_adsb-feed_1	/app/start.sh	Up	0.0.0.0:9000->9000/tcp,:::9000->9000/tcp
ads-b-siem_co-atc_1	/app/co-atc -config /app/c ...	Up	0.0.0.0:8000->8000/tcp,:::8000->8000/tcp
ads-b-siem_elasticsearch_1	/bin/tini -- /usr/local/bi ...	Up (healthy)	0.0.0.0:9200->9200/tcp,:::9200->9200/tcp, 9300/tcp
ads-b-siem_kibana_1	/bin/tini -- /usr/local/bi ...	Up	0.0.0.0:5601->5601/tcp,:::5601->5601/tcp
ads-b-siem_logstash_1	/usr/local/bin/docker-entr ...	Up (healthy)	0.0.0.0:5044->5044/tcp,:::5044->5044/tcp, 0.0.0.0:9600->9600/tcp,:::9600->9600/tcp

Figura 20. Despliegue y verificacion de servicios mediante Docker Compose.

Una vez en marcha, el servicio `adsb-feed` comienza a descargar datos reales desde

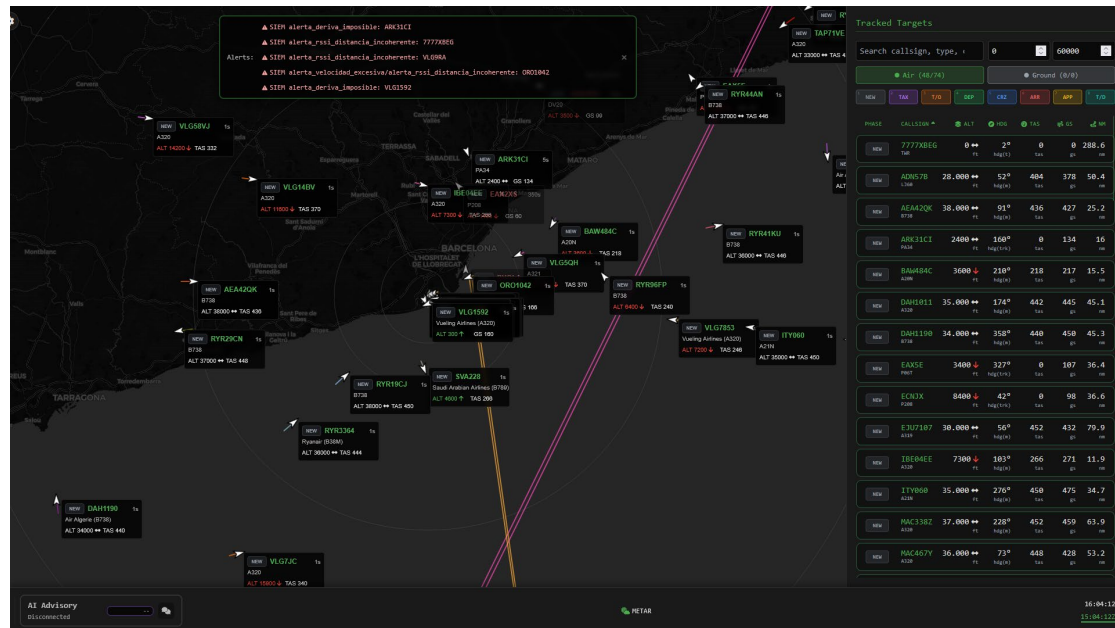


Figura 23. Alertas que indica Co-ATC (rodeadas): SIEM alerta_deriva_imposible: ARK31CI && SIEM alerta_rssi_distancia_incoherente: 7777XBEG.

Desde el punto de vista del usuario, Co-ATC muestra un trafico aparentemente normal, pero con aeronaves que presentan parametros fisicamente improbables o imposibles. Esta situacion emula un ataque o una interferencia sobre los datos ADS-B y permite evaluar si el sistema lo detecta correctamente.

El proceso completo ocurre en tiempo casi real. El feed genera el `aircraft.json` con el avion modificado y Co-ATC lo incorpora como parte del estado general de la aeronave.

A continuacion, Logstash consulta el endpoint `/api/v1/aircraft`, separa los datos por avion y aplica las reglas de deteccion. En este caso, se generan dos etiquetas: `alerta\_velocidad\_excesiva` y `alerta\_rssi\_distancia\_incoherente`. El evento resultante se indexa en Elasticsearch bajo el indice diario `adsb-siem-YYYY.MM.dd`, quedando inmediatamente disponible para consulta y visualizacion.

Por ello, el siguiente paso es comprobar que Logstash esta recogiendo datos desde Co-ATC y enviandolos a Elasticsearch. Para ello, se revisa su log con `docker-compose logs -f logstash`.

6.2. Deteccion en Kibana (Discover)

Desde Kibana, el operador puede verificar la anomalia usando *Discover*.

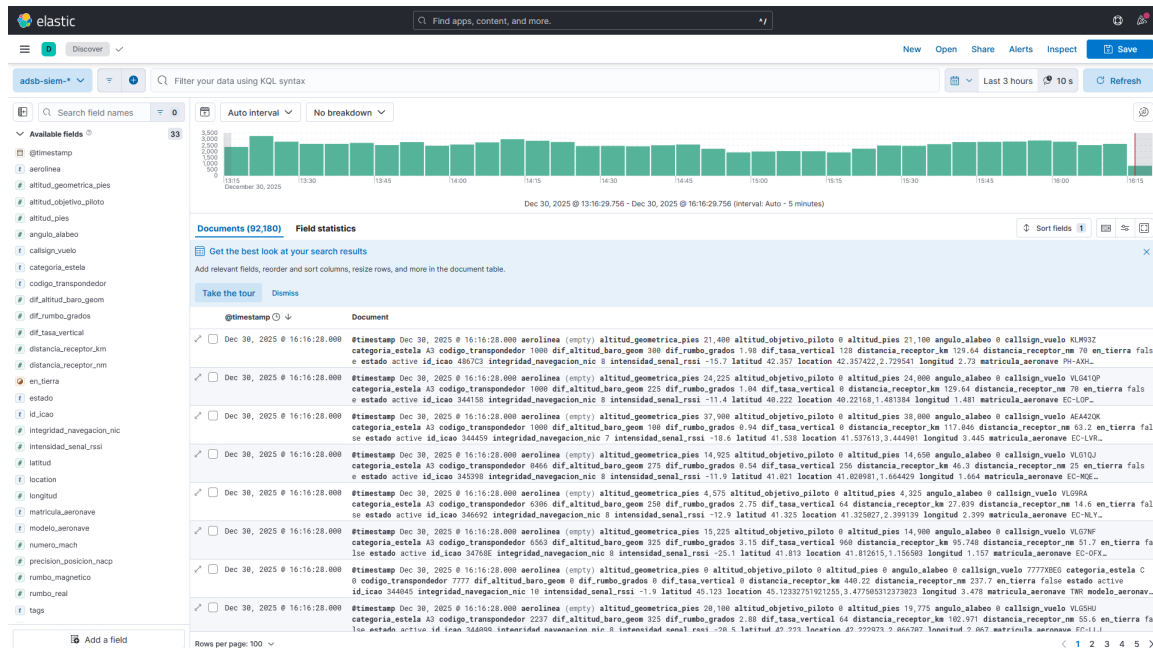


Figura 24. Interfaz *Discover* de Kibana para la inspeccion y filtrado de eventos ADS-B normalizados.

Al seleccionar el *data view* `adsb-siem-*` y aplicar un filtro doble KQL como `tags: "alerta\velocidad\_excesiva"`, se muestran los eventos anómalos generados por estas dos anomalías en los últimos minutos.

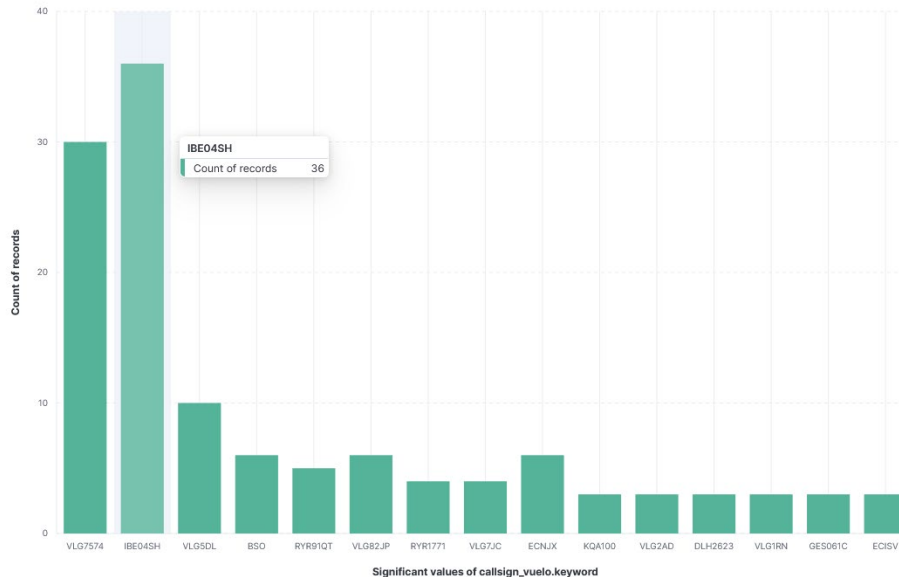


Figura 25. Grafico de barras que muestra el numero de eventos por *callsign* (`callsign_vuelo.keyword`) dentro del rango temporal seleccionado en Kibana.

Cada barra representa un avion o *callsign* y su altura indica cuantos registros se han generado para ese vuelo. En el ejemplo, IBE04SH es el *callsign* con mas eventos (36), seguido por VLG7574 (~30), mientras que el resto tiene presencia mucho menor.

Volviendo a las alertas de Co-ATC, para verificar que el stack funciona correctamente se comprueba

mediante filtros KQL de anomalias (etiquetadas por Logstash) e introduciendo los *callsigns* de las aeronaves que indica el frontend Co-ATC.

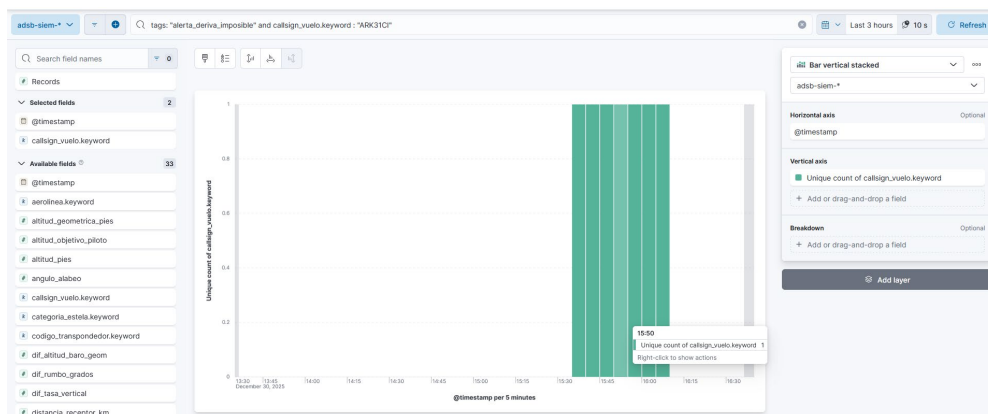


Figura 26. Grafico de barras en Kibana para la monitorizacion temporal de alertas especificas por aeronave.

Esta vista permite validar rapidamente la coherencia del evento y confirmar que los datos que el SIEM analiza coinciden con lo que Co-ATC esta mostrando.

La evidencia confirma que el stack funciona de extremo a extremo. Kibana muestra eventos agregados en tiempo real sobre el indice `adsb-siem-*`, demostrando que la ingesta y el procesamiento en Logstash son correctos. A la vez, Co-ATC visualiza las alertas SIEM generadas, lo que verifica que las reglas se aplican y que el sistema notifica al operador. En conjunto, se valida el flujo completo desde el feed ADS-B hasta la deteccion y visualizacion operativa de anomalias.

7. Relacion con la asignatura y resumen de lo realizado

En esta practica se ha construido un laboratorio completo que implementa un SIEM funcional sobre trafico ADS-B real. El sistema se despliega con Docker Compose e integra cinco servicios principales: `adsb-feed`, Co-ATC, Logstash, Elasticsearch y Kibana.

Desde el punto de vista de la asignatura, el trabajo conecta con varios bloques. En administracion de servicios, el laboratorio exige arrancar y detener servicios, revisar su estado y gestionar logs por contenedor, lo que reproduce tareas reales en algunas plataformas. En monitorizacion y gestion de logs, Logstash, Elasticsearch y Kibana representan una canalizacion completa: ingesta estructurada, transformacion, deduplicacion, indexacion y visualizacion, equivalente a un sistema de *logging* avanzado. En seguridad, las reglas SIEM implementadas (altitud negativa, velocidad excesiva, spoofing por RSSI/NIC, squawk de emergencia, etc.) justifican la deteccion basada en correlacion y anomalias, alineada con los contenidos de auditoria y respuesta ante incidentes. En redes, el proyecto utiliza APIs HTTP internas, expone puertos y maneja un flujo de datos distribuido entre contenedores. Y en virtualizacion o contenerizacion, Docker permite aislar servicios, asegurar reproducibilidad y simplificar el despliegue, que es una competencia directa del temario.

En terminos de resultados, se demuestra que el stack funciona de extremo a extremo: el trafico real

se refleja en Co-ATC, las anomalías se detectan en Logstash, quedan indexadas en Elasticsearch, se visualizan en Kibana y aparecen como alertas operativas en la interfaz. Esto valida tanto la implementación técnica como la conexión con los objetivos formativos: administración de sistemas, gestión de servicios, seguridad, monitorización y operación de infraestructuras.

8. Conclusiones y pasos siguientes

La realización de este proyecto ha permitido validar con éxito la hipótesis inicial de que Elastic Stack constituye una solución viable, flexible y potente para actuar como un sistema de Gestión de Eventos e Información de Seguridad (SIEM) en entornos críticos como la aviación, aportando una capa de seguridad necesaria a protocolos vulnerables por diseño como el ADS-B. A diferencia de la propuesta teórica inicial, esta entrega final ha logrado crear un laboratorio funcional y reproducible mediante contenedores Docker, capaz de transformar una ingesta de datos de tráfico aéreo real en información operativa de seguridad.

Uno de los pilares fundamentales del éxito de esta implementación ha sido el diseño de una arquitectura modular donde Logstash no se limita a la mera recolección de logs, sino que actúa como un motor de procesamiento lógico avanzado. Al trasladar la lógica de detección de amenazas a la fase de ingesta, el sistema es capaz de evaluar en tiempo casi real la consistencia física de los datos, identificando anomalías críticas como velocidades imposibles, altitudes negativas o incoherencias entre la intensidad de la señal (RSSI) y la distancia, las cuales suelen ser indicativas de ataques de spoofing o errores de integridad. La capacidad del servicio de adquisición desarrollado para inyectar anomalías controladas sobre tráfico legítimo ha servido para verificar empíricamente la eficacia de estas reglas, demostrando que el sistema puede distinguir eventos maliciosos dentro de un entorno de operaciones normales sin generar falsos positivos excesivos.

Asimismo, el proyecto ha logrado cerrar el ciclo de inteligencia de seguridad mediante la integración efectiva del SIEM con la interfaz operativa del controlador en Co-ATC, simulando este el software corporativo de cualquier sistema de control aéreo real. Esta característica trasciende la administración de sistemas tradicional, ya que no solo almacena y visualiza datos en los dashboards de Kibana para el análisis forense, sino que notifica las alertas directamente en el radar del operador en el momento en que ocurren. Este enfoque demuestra como las herramientas *open source* de monitorización pueden converger con los sistemas de operación en tiempo real, mejorando la conciencia situacional y la capacidad de respuesta ante incidentes sin obligar al usuario a cambiar de contexto visual.

Desde una perspectiva académica, el desarrollo de este laboratorio ha supuesto la aplicación práctica e integrada de las competencias clave de la asignatura, abarcando desde la virtualización y orquestación de servicios con Docker Compose hasta la gestión avanzada de logs estructurados y la seguridad de la información. La infraestructura resultante cumple con los requisitos funcionales de reproducibilidad y modularidad, permitiendo aislar cada componente del stack para facilitar su mantenimiento y escalabilidad. En definitiva, este trabajo demuestra que es posible mitigar las vulnerabilidades inherentes a las comunicaciones ADS-B no cifradas mediante una arquitectura de monitorización inteligente y centralizada, consolidando a Elastic Stack

como una herramienta idonea para la deteccion de intrusiones y la gestion de incidentes en infraestructuras criticas simuladas.

9. Referencias y bibliografia

1. Elastic. (2023, 17 de abril). *Flightwatching helps carriers reduce CO₂ emissions and save \$100,000+ per aircraft per year, with Elastic*. <https://www.elastic.co/blog/flightwatching-helps-carriers-with-elastic>
2. La Roche, A., & Beeching, B. (2025, 18 de marzo). *Elasticsearch in the aviation industry: A game-changer for data management*. Elastic. <https://www.elastic.co/blog/elasticsearch-data-management-aviation>
3. Segurilatam. (2017, 18 de mayo). *Ciberseguridad en aeropuertos y companias de aviacion civil*. https://www.segurilatam.com/seguridad-por-sectores/puertos-y-aeropuertos/ciberseguridad-en-aeropuertos-y-companias-de-aviacion-civil_20170518.html
4. Controladores Aereos. (2024, 8 de enero). *ADS-B: La Revolucion en la Vigilancia del Trafico Aereo en Espana*. <https://controladoresaereos.es/ads-b-la-revolucion-en-la-vigilancia-del-trafico-aereo-en-espana/>
5. Murisa, W. (2025). *A Conceptual SOC Framework for Air Traffic Management Systems*. Proceedings of ICISSP 2025. Scitepress. <https://www.scitepress.org/Papers/2025/131742/131742.pdf>
6. Kosho. (s. f.). *Flight Track - ADS-B Signal Visualization with Elastic Stack* [Repositorio GitHub]. <https://github.com/kosho/flight-track>
7. Gilbert, G. (2017, 27 de febrero). *FAA Cautions about Transponder and ADS-B Testing*. AINonline. <https://www.ainonline.com/aviation-news/aerospace/2017-02-27/faa-cautions-about-transponder-and-ads-b-testing>
8. Ahmed, W., Bhatti, N. A., Masood, A., Alharbi, A. A., & Alotaibi, A. (2024). *Advancements in ADS-B Security: A Comprehensive Survey of Vulnerabilities, Mitigation Strategies, System Requirements and Emerging Research Trends*. Preprints.org. <https://doi.org/10.20944/preprints202405.0586.v1>
9. Manzoor, J. (2024). *Evaluating security and performance of open-source SIEM systems*. PMC. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10977669/>
10. Sheeraz, M. (2024). *Revolutionizing SIEM Security: An Innovative Correlation Engine for Multi-Layered Attack Detection*. *Sensors*, 24(15), 4901. MDPI. <https://doi.org/10.3390/s24154901>
11. Law, Y. W., & Slay, J. (2022). *SIEM4GS: Security Information and Event Management for a Virtual Ground Station Testbed*. University of South Australia / SmartSat CRC. <https://ywlaw.github.io/pub/law2022siem4gs.pdf>

12. Aymard Cuello, M. (2019). *Security Monitoring System Applied to IoT*. Tesis de master, Universidad Politecnica de Madrid. https://oa.upm.es/65634/1/TESIS_MASTER_MARIANNE_AYMARD_CUELLO.pdf
13. Ying, X., Mazer, J., Bernieri, G., Conti, M., Bushnell, L., & Poovendran, R. (2019). *Detecting ADS-B Spoofing Attacks using Deep Neural Networks*. arXiv. <https://arxiv.org/abs/1904.09969>
14. Ngamboe, M., Marrocco, J.-S., Ouattara, J.-Y., Fernandez, J. M., & Nicolescu, G. (2023). *CABBA: Compatible Authenticated Bandwidth-efficient Broadcast protocol for ADS-B*. arXiv. <https://arxiv.org/abs/2312.09870>
15. Akerman, S., Habler, E., & Shabtai, A. (2019). *VizADS-B: Analyzing Sequences of ADS-B Images Using Explainable Convolutional LSTM Encoder-Decoder to Detect Cyber Attacks*. arXiv. <https://arxiv.org/abs/1906.07921>
16. Elastic. (s. f.). *AI-driven SIEM that is open source and affordable*. <https://www.elastic.co/security/siem>